

Homework 5 - Recursion

Course: CIS 284 - Computer Programming 2 - Fall 2026

Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: Oct 8 at 11:59pm

Zip up all java files and upload to Canvas. **Make sure each file has your name at the top**

For 2 bonus points, answer the following questions (as a **comment** on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear in the directions? Do you have any other feedback?

Your responses will be helpful feedback for refining these assignments in future courses.

Assignment Overview

This assignment will help you practice writing recursive functions and drawing fractals.

Grading Information

As this homework emphasizes recursion, your solutions will be graded both on correctness and whether they properly use recursion. As a rule of thumb, there should not be any loops. Pay close attention to the specific guidelines given in each part. *If you can't complete everything, it is better to submit partial work than miss the deadline.*

Points	Name	Description
25	Effort	Do the programs compile and run? Was a reasonable attempt made?
5	Style	Is the code formatted well? Is it readable? Are variables named well?
15	Part 1	Correct recursive implementation of reverse function.
20	Part 2	Correct recursive implementation of sum function.
25	Part 3	Correct recursive implementation to draw the Sierpinski square.
10	Part 4	Creative fractal exercise.

Note: Your output does not need to match my examples word for word. Feel free to add additional messages or personalize it, as long as it satisfies the requirements.

1. Reverse

In this part, you will write a recursive function that can reverse a String.

For example:

- Given "abc", print cba
- Given "hello", print olleh
- Given "This is fun!", print !nuf si sihT

Requirements:

- Your program file must be called **Reverse.java**
- Your function should match the given signature: **String reverse(String s)**
- Your function must be recursive (no loops!)
- **Hint:** model your solution after the **isPalindrome** example in the slides

Your program should do the following:

1. Read 1 **line** from standard in (the string may contain spaces).
2. Call the function **reverse(line)**
3. Print the result

Output Example:

```
java Reverse.java
-----
Please enter some text:
a man a plan a canal panama

Your text reversed is:
amanap lanac a nalp a nam a
```

2. Recursive Sum

In this part, you will implement a function that sums over an array using recursion.

For example:

- Given [2, 4, 6], the sum is 12
- Given [10, -3, 0, 2] the sum is 9

Requirements:

- Your program must be named **Sum.java**
- Your function should match the given signature: **int sum(int[] arr)**
- It should call a different recursive function that takes a range (**lo - hi**)
- Compute the sum of that range using recursion (no loops!)
 - Determine the base cases and return their values.
 - Otherwise, split the array into two equal halves, then sum each half and combine
- **Hint:** review the **binarySearch** example code for a guide

Your program should do the following:

1. Read 1 int from standard in (the number of integers to follow)
2. Read **n** integers from standard in, into an array
3. Call the function **sum(arr)**
4. Print the result

Output Example:

```
java Sum.java
-----
Enter the number of integers given:
5

Enter 5 integers
2
-10
5
4
-3

The sum is -2
```

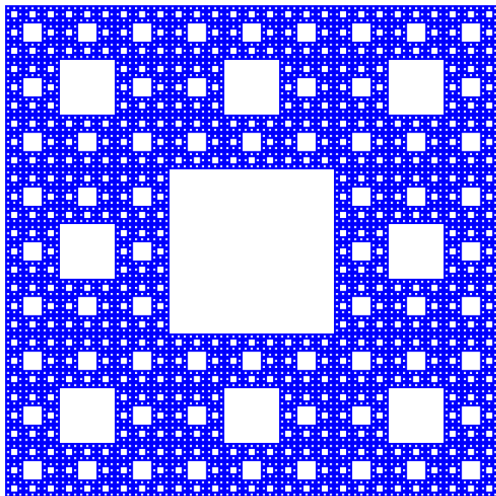
Note: I have provided a test case file called **sum_test1.txt**. You can use this test by typing:

```
cat sum_test1.txt | java Sum.java
```

I recommend you make additional test files

3. Sierpinski Carpet

We covered the Sierpinski Triangle in class. But there is another closely related fractal called the Sierpinski Square, or Sierpinski Carpet. It works in a similar way, but uses squares instead.

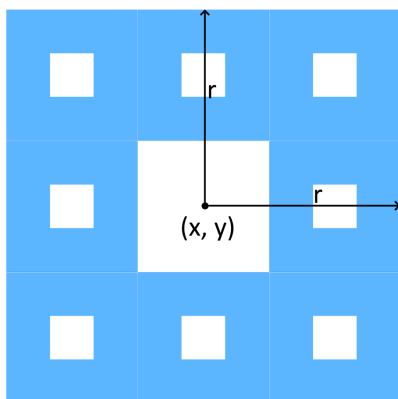


Requirements:

- Create a file called **SierpinskiSquare.java**
- It should take 1 **optional** command line argument: an int (depth to draw)
- If no command line argument is given, use default of 5
- Draw the fractal as shown above with the given depth
- Your solution should use double buffering

I highly recommend that you refer to the **L13_Rec_Drawing/Sierpinski.java** example.

As a hint, model the larger square using a center x, y coordinate, and a 'radius'.



Think about how many recursive calls (smaller squares) you will need to make, and what the position/size of each one will be.

4. Creative Exercise

For this part, you are given free rein to create your own fractal. You may start with one of the given fractals covered in class and modify it, or design your own. Feel free to play around with color, positioning, mixing up shapes, or other ways to creatively divide into smaller pieces.

This part is worth 10 points, which will be allocated based on the level of creativity and effort shown. This is not intended to be a large assignment. Even one significant modification will receive good share of the credit.